SuperSec: desenvolvimento de um sistema web de pedidos de produtos de limpeza

Resumo. Este trabalho apresenta o desenvolvimento do SuperSec, um sistema web destinado à comercialização simplificada de produtos de limpeza. A solução permite cadastro e autenticação de usuários, consulta de produtos, montagem de carrinho, finalização de pedidos, atualização de estoque e consulta do histórico de compras. O backend foi implementado em Java com Spring Boot, o frontend em React e a persistência em PostgreSQL. A autenticação utiliza Spring Security, senhas protegidas por BCrypt e tokens JWT. A aplicação é executada em containers Docker, proporcionando um ambiente reproduzível para demonstração acadêmica.

Palavras-chave: sistemas web; comércio eletrônico; React; Spring Boot; PostgreSQL; Docker; JWT.

1 Introdução

Sistemas de pedidos integram interface, regras de negócio, autenticação e persistência de dados. Mesmo uma compra simples exige que o sistema identifique o usuário, controle a quantidade disponível, registre os itens selecionados e preserve o histórico da operação. Quando essas responsabilidades ficam distribuídas entre diferentes camadas, a consistência dos dados passa a ser um requisito central.

O SuperSec foi desenvolvido como um protótipo acadêmico de uma loja virtual especializada em produtos de limpeza. O escopo foi definido para demonstrar o ciclo essencial de compra sem depender de serviços externos de pagamento: acesso ao catálogo, criação de conta, autenticação, carrinho, checkout, atualização de estoque e consulta dos pedidos realizados.

2 Objetivos

O objetivo geral é construir um sistema web funcional de pedidos, aplicando conceitos de engenharia de software, programação orientada a objetos, banco de dados, segurança e implantação. Como objetivos específicos, buscou-se: implementar uma API REST; persistir usuários, produtos e pedidos em PostgreSQL; proteger credenciais com BCrypt; utilizar JWT para autenticação; validar estoque no servidor; oferecer uma interface responsiva; e disponibilizar o ambiente por Docker Compose.

3 Metodologia e arquitetura

A solução foi organizada em três componentes principais. O frontend, desenvolvido em React com Vite, concentra a apresentação e a interação com o usuário. O backend, desenvolvido em Java 21 e Spring Boot, disponibiliza endpoints REST para autenticação, produtos e pedidos. O PostgreSQL armazena os dados persistentes. A comunicação entre frontend e backend ocorre por HTTP utilizando JSON.

Camada	Tecnologia	Responsabilidade
Interface	React + Vite	Catálogo, busca, carrinho, autenticação e histórico
API	Java 21 + Spring Boot	Regras de negócio, autenticação e pedidos
Persistência	PostgreSQL + JPA/Hibernate	Usuários, produtos, pedidos e itens
Segurança	Spring Security + JWT + BCrypt	Autenticação e autorização
Infraestrutura	Docker Compose	Execução integrada dos serviços

A persistência utiliza entidades relacionadas: um usuário pode possuir vários pedidos; cada pedido possui itens; e cada item referencia um produto e registra o preço praticado no momento da compra. Essa modelagem permite manter o histórico mesmo quando o preço ou o estoque atual do produto é alterado.

4 Desenvolvimento

O fluxo começa no catálogo público. Os produtos são carregados pela API e apresentados com nome, descrição, preço, estoque e imagem. O usuário pode pesquisar e ordenar o catálogo, visualizar detalhes e adicionar produtos ao carrinho. O carrinho é mantido no navegador para preservar os itens durante a navegação.

Para finalizar uma compra, o usuário precisa estar autenticado. O frontend envia ao backend somente os identificadores dos produtos e as quantidades. O servidor consulta novamente os produtos no banco, verifica se

estão ativos e se há estoque suficiente. Essa validação é feita no backend para evitar que regras críticas dependam exclusivamente da interface.

Após as validações, o sistema cria o pedido, registra os itens, calcula o subtotal e aplica a regra de frete definida no projeto. Em seguida, reduz o estoque dos produtos. A operação é transacional, de modo que uma falha durante o processamento impede a persistência incompleta do pedido.

5 Autenticação e segurança

No cadastro, a senha recebida não é armazenada diretamente: o sistema aplica BCrypt antes de persistir o usuário. No login, as credenciais são verificadas e um token JWT é emitido. As requisições protegidas enviam o token no cabeçalho Authorization. Operações administrativas utilizam autorização por papel, restringindo o gerenciamento de produtos aos usuários com perfil ADMIN.

A configuração de segurança também utiliza sessão stateless, CORS para a origem do frontend e proteção das rotas de pedidos. O projeto não pretende representar um ambiente de produção completo: a chave JWT e a senha administrativa fornecidas no material são exclusivamente para demonstração acadêmica e devem ser substituídas em um cenário real.

6 Banco de dados e regras de negócio

O PostgreSQL contém as tabelas users, products, orders e order_items. A estrutura SQL definitiva está documentada em docs/schema.sql, enquanto database/init.sql fornece dados demonstrativos do catálogo de produtos de limpeza. No Docker Compose, esses arquivos são executados na inicialização de um banco novo, e o Hibernate mantém o mapeamento das entidades.

Entre as restrições de integridade estão chaves primárias, relacionamentos por chave estrangeira, e-mails únicos e verificações para impedir preço ou estoque negativos. O pedido armazena o preço unitário de cada item, preservando o valor histórico da compra independentemente de alterações posteriores no catálogo.

7 Containerização e execução

O Docker Compose coordena três serviços: PostgreSQL, backend e frontend. O banco possui healthcheck e o backend depende da disponibilidade do banco. O backend utiliza o perfil de configuração fornecido pelas variáveis de ambiente do Compose. O frontend é servido pelo container próprio e acessado pelo navegador em http://localhost:5173, enquanto a API fica disponível em http://localhost:8080.

Essa organização reduz a quantidade de configuração manual necessária para a demonstração. O projeto também mantém os diretórios separados por responsabilidade, facilitando versionamento e manutenção.

8 Resultados e testes

O sistema implementa o fluxo completo proposto: cadastro, login, catálogo, carrinho, confirmação da compra, atualização de estoque e histórico de pedidos. Após a finalização, o usuário recebe uma tela de confirmação com mensagem de agradecimento e pode acessar diretamente a área de pedidos. O histórico consulta os pedidos associados ao usuário autenticado.

Também foi incluída uma área administrativa para cadastrar, editar, ativar e desativar produtos. Produtos inativos deixam de aparecer no catálogo público, mas permanecem registrados no banco. O roteiro de demonstração do README orienta a execução dos principais testes funcionais e a verificação do estado persistido no PostgreSQL.

9 Limitações

O projeto não processa pagamentos reais, não emite documentos fiscais e não possui integração logística. As imagens dos produtos são referenciadas por URLs externas e o catálogo utiliza uma estratégia visual de fallback caso uma imagem não esteja disponível. Essas limitações foram mantidas para preservar o escopo acadêmico e concentrar a demonstração nas funcionalidades de engenharia de software.

10 Conclusão

O desenvolvimento do SuperSec integrou conhecimentos de programação, desenvolvimento web, banco de dados, segurança e implantação. A separação entre frontend, backend e persistência permitiu implementar o fluxo de pedidos de forma organizada, enquanto a validação de estoque e o registro do preço no momento da compra demonstram a aplicação de regras de negócio no servidor.

Como evolução futura, o sistema pode receber gateway de pagamento, recuperação de senha, notificações, filtros avançados, acompanhamento de entrega e recursos administrativos adicionais. Para o objetivo acadêmico definido, entretanto, a versão entregue contempla o ciclo principal de compra, persistência, autenticação, segurança básica e execução containerizada.

Referências

SPRING. *Spring Boot Reference Documentation*. Disponível em: https://docs.spring.io/spring-boot/. Acesso em: 30 set. 2026.

SPRING. *Spring Security Reference*. Disponível em: https://docs.spring.io/spring-security/reference/. Acesso em: 30 set. 2026.

REACT. *React Documentation*. Disponível em: https://react.dev/. Acesso em: 30 set. 2026.

POSTGRESQL. *PostgreSQL Documentation*. Disponível em: https://www.postgresql.org/docs/. Acesso em: 30 set. 2026.

DOCKER. *Docker Documentation*. Disponível em: https://docs.docker.com/. Acesso em: 30 set. 2026.